

Teoria della dimostrazione, Teoria dei tipi, Proof Assistants e AI

verso una nuova pratica della matematica

Marino Miculan

Scuola Estiva di Logica AILA 2026

Bardonecchia, 31 agosto – 5 settembre 2026

DMIF, Università di Udine | marino.miculan@uniud.it | <https://marino.miculan.org>

LEZIONE 2

Polimorfismo e il λ -cubo di Barendregt: verso il Calcolo delle Costruzioni

AILA 2026 — Bardonecchia

Obiettivo

Mostrare che MLTT e CoC non sono sistemi isolati, ma **punti di una mappa sistematica a tre assi**: uno scheletro mentale per orientarsi in qualunque proof assistant si incontrerà in futuro.

In questa lezione

1. Lezione 2 — λ -cubo
 - 1.1 Quantificazione universale sui tipi
 - 1.2 Espressività: le codifiche di Church
 - 1.3 L'altra strada: quantificare sugli individui
 - 1.4 Tre assi di generalizzazione
 - 1.5 Il cubo delle logiche
 - 1.6 Pure Type Systems
 - 1.7 Il vertice: il Calcolo delle Costruzioni
 - 1.8 Il paradosso di Girard, e come lo si evita

Blocco 1

System F (λ^2)

Dal problema all'idea

In STLC la funzione identità va scritta una volta per ogni tipo:

$$\lambda x:A. x : A \rightarrow A \quad \lambda x:B. x : B \rightarrow B \quad \dots$$

Dal problema all'idea

In STLC la funzione identità va scritta una volta per ogni tipo:

$$\lambda x:A. x : A \rightarrow A \quad \lambda x:B. x : B \rightarrow B \quad \dots$$

System F (Girard 1972, Reynolds 1974) internalizza il «per ogni tipo»:

$$\begin{aligned} A, B &::= X \mid A \rightarrow B \mid \forall X. A \\ t, u &::= x \mid \lambda x:A. t \mid t u \mid \Lambda X. t \mid t[A] \end{aligned}$$

$$\Lambda X. \lambda x:X. x : \forall X. X \rightarrow X$$

i Lo conoscete già: è il **polimorfismo**

In Haskell `id :: a -> a` è $\forall X. X \rightarrow X$: il **forall** resta implicito, e le applicazioni di tipo le inferisce il compilatore.

ML e Haskell: quanto System F ci sta dentro?

Il *Core* di GHC è una variante di System F: ΛX e $t[A]$ ci sono, li inserisce il compilatore.

RANGO — dove possono stare i $\forall$

1 (*prenex*): $\forall X. X \rightarrow X$

ML, e Haskell di default

2: $(\forall X. X \rightarrow X) \rightarrow \mathbb{N}$

Haskell con **RankNTypes**, ma va annotato

Sistema	Inferenza dei tipi
Hindley–Milner (rango 1)	decidibile, con <i>tipi principali</i> (Damas–Milner 1982)
System F di rango 2	ancora decidibile; dal rango 3, no
System F pieno, alla Curry	indecidibile (Wells 1994)

Il punto

Alla **Church** — con Λ e $t[A]$ scritti a mano, come faremo noi — il type checking è immediato: è la scelta dei proof assistant.

$$\frac{\Gamma, X \vdash t : A \quad X \notin \text{FV}(\Gamma)}{\Gamma \vdash \lambda X. t : \forall X. A} \quad \forall I$$

$$\frac{\Gamma \vdash t : \forall X. A}{\Gamma \vdash t [B] : A[X := B]} \quad \forall E$$

$$(\lambda X. t) [B] \longrightarrow_{\beta} t[X := B]$$

Lettura logica

È la logica proposizionale intuizionista del *second'ordine*: si quantifica su proposizioni. La condizione $X \notin \text{FV}(\Gamma)$ è la consueta *eigenvariable condition*.

Tutto è una funzione

I TIPI

$\text{Bool} := \forall X. X \rightarrow X \rightarrow X$

$\text{Nat} := \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$

$\text{List } A := \forall X. (A \rightarrow X \rightarrow X) \rightarrow X \rightarrow X$

I COSTRUTTORI — sono i termini che *scelgono* un argomento

$\text{true} := \Lambda X. \lambda x:X. \lambda y:X. x$

$\text{zero} := \Lambda X. \lambda f:X \rightarrow X. \lambda z:X. z$

$\text{false} := \Lambda X. \lambda x:X. \lambda y:X. y$

$\text{succ} := \lambda n:\text{Nat}. \Lambda X. \lambda f. \lambda z. f (n [X] f z)$

L'ELIMINAZIONE — basta *applicare* il dato

$\text{if } b \text{ then } u \text{ else } v := b [C] u v \quad \underline{n} [C] f z \rightarrow_{\beta} f^n(z)$

Perché proprio quei tipi? Il più piccolo insieme chiuso

Al second'ordine, $\text{Nat} = \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$ dice:

n appartiene a ogni insieme X che contenga lo zero e sia chiuso rispetto al successore.

I due costruttori si impacchettano in una sola mappa:

$$\underbrace{(X \rightarrow X)}_{\text{successore}} \rightarrow \underbrace{X}_{\text{zero}} \rightarrow X \quad \cong \quad (X + \mathbf{1} \rightarrow X) \rightarrow X$$
$$\text{Nat} = \bigcap \{X : X + \mathbf{1} \rightarrow X\}$$

💡 Il più piccolo insieme chiuso rispetto ai costruttori

Un'intersezione di insiemi chiusi è chiusa ed è contenuta in tutti: è dunque **il minimo**.
È la definizione di $\mathbb{N}$ di Dedekind (1888), scritta come tipo.

In generale $T := \forall X. (F(X) \rightarrow X) \rightarrow X$, con $F(X) = X + \mathbf{1}$ per Nat : categorialmente, l'algebra iniziale di F .  L3: si ricava l'iterazione, non l'induzione

Fin dove arriva System F?

Sistema	Funzioni $\mathbb{N} \rightarrow \mathbb{N}$ rappresentabili
$\lambda_{\rightarrow}$ (STLC)	i <i>polinomi estesi</i> — pochissimo
Sistema T di Gödel	le totali dimostrabili in PA (prim'ordine)
System F	le totali dimostrabili in PA₂ (second'ordine)

Girard 1972 (GLT, cap. 15); fra queste c'è Ackermann, che non è ricorsiva primitiva.

⚠ Il paradosso apparente

La classe è enorme, eppure **ogni** termine tipabile termina. Ma la SN di System F non si dimostra *dentro* PA₂: da essa segue Con(PA₂), e per il secondo teorema di Gödel nessuna teoria coerente così forte dimostra la propria coerenza.

Con(PA₂) è l'enunciato *aritmetico* che, via numerazione di Gödel, dice «PA₂ non dimostra $0 = 1$ ». Girard aggira l'ostacolo con i *candidati di riducibilità*: impredicativi.

⚠ Il punto delicato

In $\forall X.A$ la variabile X spazia su *tutti* i tipi — incluso $\forall X.A$ stesso. La definizione è **impredicativa**: definisce un oggetto quantificando su una totalità che lo contiene.

- ▶ non è un difetto: è ciò che dà a System F la sua espressività;
- ▶ **ma** è incompatibile con universi «ingenui»: l'assunzione $\text{Type} : \text{Type}$ rende il sistema incoerente (**paradosso di Girard**);
- ▶ due risposte diverse:
 - ▶ MLTT: *gerarchia predicativa* di universi;
 - ▶ CoC/CIC: impredicatività confinata a un unico universo di *proposizioni*, Prop .

Blocco 2

Il λ -cubo

Tipi che dipendono da termini: λP

L'altra strada lasciata aperta a fine L1: quantificare non sui *tipi*, ma sugli **individui**.

SINTASSI — una sola aggiunta

$A, B ::= \dots \mid \forall x:A. B$ es. $\forall n:\mathbb{N}. \text{Vec } n \rightarrow \text{Vec } (n+1)$

$$\frac{\Gamma, x:A \vdash t : B}{\Gamma \vdash \lambda x:A. t : \forall x:A. B} \forall I \qquad \frac{\Gamma \vdash t : \forall x:A. B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B[x := u]} \forall E$$

Se x non compare in B , $\forall x:A. B$ è $A \rightarrow B$. In L3 si chiamerà $\Pi x:A. B$.

💡 Perché B possa parlare di x serve una novità

Un **predicato** è una famiglia di tipi indicizzata da termini: $\text{Vec} : \mathbb{N} \rightarrow \text{Tipo}$. **La vera aggiunta di λP è poter formare quel $\mathbb{N} \rightarrow \text{Tipo}$.**

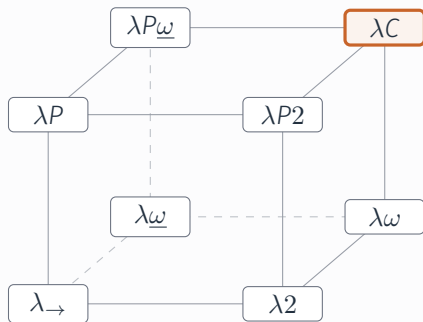
Che cosa può dipendere da che cosa

Partendo da $\lambda_{\rightarrow}$ (termini che dipendono da termini), tre direzioni indipendenti:

Dipendenza	Sistema	Che cos'è
termini da tipi	λ_2	polimorfismo (System F)
tipi da tipi	$\lambda_{\underline{\omega}}$	operatori di tipo, costruttori
tipi da termini	λ_P	tipi dipendenti

Le tre direzioni sono **ortogonali**: si possono combinare liberamente, ottenendo $2^3 = 8$ sistemi.

Il λ -cubo di Barendregt



$\rightarrow$ termini da tipi

$\uparrow$ tipi da termini

$\nearrow$ tipi da tipi

λC = Calcolo delle Costruzioni: tutte e tre le dimensioni.

Il cubo, letto come logica

Curry–Howard non si ferma alla logica proposizionale: ogni asse del cubo aggiunge un pezzo preciso di **potere logico**.

Asse	Nel calcolo	Nella logica
$\lambda 2$	termini da tipi	quantificazione sulle <i>proposizioni</i> (2° ordine)
$\lambda \omega$	tipi da tipi	<i>costruttori</i> di proposizioni (ordine superiore)
λP	tipi da termini	predicati e quantificazione sugli <i>individui</i>

Nota

Sono logiche **minimali intuizioniste**: i soli connettivi primitivi sono $\rightarrow$ e $\forall$. Gli altri o si definiscono al second'ordine $\otimes : A \vee B := \forall X. (A \rightarrow X) \rightarrow (B \rightarrow X) \rightarrow X$, o si aggiungono come costanti.

Il cubo delle logiche di Barendregt

Sistema	Logica	
$\lambda_{\rightarrow}$	PROP	proposizionale (minimale)
λ_2	PROP2	proposizionale del 2° ordine
$\lambda_{\underline{\omega}}$	PROP $\underline{\omega}$	... debolmente di ordine superiore
λ_{ω}	PROP ω	... di ordine superiore
λP	PRED	dei predicati (prim'ordine) — è la base di LF
λP_2	PRED2	dei predicati del 2° ordine
$\lambda P_{\underline{\omega}}$	PRED $\underline{\omega}$	... debolmente di ordine superiore
λC	PRED ω	dei predicati di ordine superiore (HOL)

Le immersioni sono *complete*: una formula è dimostrabile se e solo se il tipo corrispondente è abitato (Geuvers 1993).

λP **rappresenta** le altre logiche — è il nucleo di LF  L1: Harper, Honsell e Plotkin, 1987;
 λC invece le **contiene**.

Un meta-framework: i PTS

Un **Pure Type System** è una tripla $(\mathcal{S}, \mathcal{A}, \mathcal{R})$:

- ▶ $\mathcal{S}$: le *sorti* (es. $\{*, \square\}$);
- ▶ $\mathcal{A} \subseteq \mathcal{S}^2$: gli *assiomi* (es. $* : \square$);
- ▶ $\mathcal{R} \subseteq \mathcal{S}^3$: le *regole* di formazione dei prodotti.

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, x:A \vdash B : s_2 \quad (s_1, s_2, s_3) \in \mathcal{R}}{\Gamma \vdash \Pi x:A. B : s_3} \text{PROD}$$

i Come si leggono le due sorti

$*$ è **Type**, il tipo dei tipi: $\mathbb{N} : *, \mathbb{N} \rightarrow \mathbb{N} : *$.

$\square$ è **Kind**, il tipo di $*$: l'assioma è $* : \square$; e con $(\square, \square)$ anche $* \rightarrow * : \square$.

In Rocq e Lean i nomi si spostano: l' $*$ di λC è Prop, e $\square$ è Type.

i Nota

Otto vertici variando *soltanto* $\mathcal{R}$: $\mathcal{S} = \{*, \square\}$ e $\mathcal{A} = \{(*, \square)\}$ restano fissi.

Sistema	$(*, *)$	$(\square, *)$	$(\square, \square)$	$(*, \square)$
$\lambda_{\rightarrow}$	✓			
λ_2	✓	✓		
$\lambda_{\underline{\omega}}$	✓		✓	
λ_{ω}	✓	✓	✓	
λ_P	✓			✓
λ_{P^2}	✓	✓		✓
$\lambda_{P\underline{\omega}}$	✓		✓	✓
λ_C	✓	✓	✓	✓

(s_1, s_2) abbrevia (s_1, s_2, s_2) . $(*, *)$ è sempre presente: è $\lambda_{\rightarrow}$.

- ▶ combina polimorfismo, operatori di tipo e tipi dipendenti;
- ▶ un'unica costruzione, il **prodotto dipendente** $\prod x:A. B$, subsume $\rightarrow$, $\forall$ e la quantificazione sui tipi;
- ▶ fortemente normalizzante e coerente;
- ▶ $\text{Prop} (= *)$ è **impredicativo**: $\forall X:\text{Prop}. X$ è ancora in Prop .

💡 Perché è il punto di arrivo naturale

Un solo binder, un solo giudizio, tre generalizzazioni — ma manca ancora qualcosa: come si definiscono $\mathbb{N}$, le liste, gli alberi? 🔗 *L3: tipi induttivi $\Rightarrow$ CIC*

Il paradosso di Girard

L'impredicatività non è gratis. Se si esagera, il sistema **collassa**: si costruisce un termine chiuso di tipo $\perp$.

- ▶ **Type : Type** — la scelta della prima teoria dei tipi di Martin-Löf (1971) — è incoerente: lo mostra Girard nella tesi del 1972;
- ▶ nel linguaggio dei PTS il colpevole è il **sistema U** , con tre sorti $*$: $\Box$: Δ e impredicatività a *due* livelli — e basta già la sua variante ridotta U^- ;
- ▶ è la versione type-theoretic di **Burali-Forti**: si costruisce l'«insieme di tutti gli ordinali» e lo si confronta con sé stesso.

La versione minimale: Hurkens 1995

Hurkens ha ridotto la costruzione a un termine di tipo $\perp$ in λU^- lungo poche righe: è la forma in cui il paradosso circola oggi, e con cui si collauda un kernel.

Che cos'è λU^-

Un PTS con **tre** sorti: $\mathcal{S} = \{*, \square, \triangle\}$, $\mathcal{A} = \{* : \square, \square : \triangle\}$.

Regola	Che cosa permette	Già in
$(*, *)$	funzioni fra tipi	$\lambda \rightarrow$
$(\square, *)$	polimorfismo: $\forall X:\square. A$ resta un <i>tipo</i>	$\lambda 2$
$(\square, \square)$	costruttori di tipo	$\lambda \omega$
$(\triangle, \square)$	$\forall X:\square. K$ resta un <i>kind</i> : impredicatività <i>al piano di sopra</i>	U^-
$(\triangle, *)$	... e produce anche un tipo	U

⚠ Dove sta il salto

λC , il vertice del cubo, ha **due** sorti e una sola regola impredicativa. λU^- ne aggiunge una terza e ripete l'impredicatività un livello più in alto: tanto basta a costruire $\perp$.

La costruzione di Hurkens

NOTAZIONE $\wp X := X \rightarrow *$ (i «sottoinsiemi» di X) $\perp := \forall p:*.p$ $\neg A := A \rightarrow \perp$

L'UNIVERSO PARADOSSALE — possibile solo con (Δ, Ω)

$$U := \forall X:\square. ((\wp \wp X \rightarrow X) \rightarrow \wp \wp X) : \square$$

$$\tau : \wp \wp U \rightarrow U$$

$$\tau t := \lambda X. \lambda f. \lambda p. t (\lambda x:U. p (f (x X f)))$$

$$\sigma : U \rightarrow \wp \wp U$$

$$\sigma s := s U (\lambda t. \tau t)$$

LA DIAGONALE

$$\Delta := \lambda y:U. \neg (\forall p:\wp U. \sigma y p \rightarrow p (\tau (\sigma y))) : \wp U$$

$$\Omega := \tau (\lambda p:\wp U. \forall x:U. \sigma x p \rightarrow p x) : U$$

Tre lemmi elementari su Δ e Ω danno un termine *chiuso* di tipo $\perp$ — in Coq è nella libreria standard, `Coq.Logic.Hurkens`.

Prima guardia: un solo universo impredicativo, in fondo

SISTEMA U — incoerente

$$* : \Box : \Delta$$

impredicativo *sia* a livello $\Box \rightarrow *$ *sia* a livello $\Delta \rightarrow \Box$: due piani sovrapposti, e il piano di sotto può parlare di quello di sopra.

CoC / CIC — coerente

$$\text{Prop} : \text{Type}_0 : \text{Type}_1 : \text{Type}_2 : \dots$$

un **solo** universo impredicativo, Prop , e sta in **fondo**; sopra, una gerarchia *predicativa* e senza cima. Non esiste $\text{Type} : \text{Type}$.

In termini di PTS, in CoC l'unica regola impredicativa è $(\Box, *)$: si può quantificare su *tutte* le proposizioni e restare una proposizione. Ma $\Box$ non è a sua volta abitante di una sorte impredicativa — ed è precisamente il secondo piano che manca al paradosso per chiudersi.

Seconda guardia: da Prop non si estraggono dati

Non basta confinare l'impredicatività: serve anche impedire che una *dimostrazione* diventi un *dato*.

- ▶ in CIC l'eliminazione da Prop verso Type è **vietata** (salvo casi singoletto): da $h : P$ con $P : \text{Prop}$ non si costruisce un valore in Type;
- ▶ senza questa restrizione l'impredicatività riapre il paradosso — Coquand, 1986;
- ▶ il contraltare: Prop è *proof-irrelevant*, e le dimostrazioni si possono cancellare in fase di estrazione.

⚠ Non è un timore teorico

In Coq **Set** era impredicativo: con certi assiomi classici diventava contraddittorio. Dalla versione 8.0 (2004) è predicativo di default, e l'impredicatività è un'opzione da chiedere esplicitamente. 🔗 L3: Lean 4 fa la stessa scelta, con *proof irrelevance* definizionale

- ▶ H. Barendregt. «**Lambda Calculi with Types**». In: *Handbook of Logic in Computer Science*. A cura di S. Abramsky, D. M. Gabbay e T. S. E. Maibaum. Vol. 2. Oxford University Press, 1992, pp. 117–309
- ▶ T. Coquand e G. Huet. «**The Calculus of Constructions**». In: *Information and Computation* 76.2–3 (1988), pp. 95–120
- ▶ J.-Y. Girard. «**Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur**». Thèse d'État. Université Paris VII, 1972 — per System F e la normalizzazione.
- ▶ A. J. C. Hurkens. «**A Simplification of Girard's Paradox**». In: *Typed Lambda Calculi and Applications (TLCA '95)*. Vol. 902. LNCS. Springer, 1995, pp. 266–278

- ▶ T. Coquand. «An Analysis of Girard's Paradox». In: *Symposium on Logic in Computer Science (LICS '86)*. IEEE, 1986, pp. 227–236
- ▶ H. Geuvers. «Logics and Type Systems». Tesi di dott. Katholieke Universiteit Nijmegen, 1993 — per il cubo delle logiche.